

Universidad Industrial de Santander
Entornos de Programación - Grupo F1¹
Investigación JWT

Jonk Keyler Sanchez Pabon¹, Alejandro Castro Rondon¹, Juan Carlos Elizalde Padilla¹

1. Conceptos Básicos

La **autenticación** es el proceso en el cual el usuario ingresa sus credenciales (por ejemplo, correo y contraseña, luego, el software debe verificar que este usuario realmente sea quien dice ser. En caso tal que las credenciales no correspondan, el sistema debe responder con *401 - No autorizado* (no se pudo demostrar la identidad). Ahora bien, si las credenciales son correctas, el sistema ha autenticado correctamente al usuario, pero el manejo de sesión se puede realizar de varias maneras, ya que, si no se implementa un mecanismo para recordar que ya hubo dicha autenticación, el usuario tendría que ingresar una y otra vez sus credenciales. Los **JSON Web Token (JWT)** es un estándar abierto (RFC 7519) que define una forma compacta y autónoma de transmitir información de forma segura entre partes como un objeto *JSON*, estos tokens son los que permiten que el sistema reconozca, acción tras acción, que el usuario autorizado es el que realmente está llevando a cabo dichas acciones. Este token *JWT* consta de tres partes codificadas en *Base64Url*, como se muestra en la *Figura 1*:

- *Header*: Indica el tipo de *token* (*JWT* en este caso) y el algoritmo de firma usado (por ejemplo HS256).
- *Payload*: Contiene los datos (*claims*) del usuario, como *sub* (subject) e información del *token* como *exp* (tiempo de expiración del token).
- *Signature*: Es la firma única que valida el *token*, esta conformada por la información del *Header*, *Payload* y una clave secreta que solo sabe el servidor.

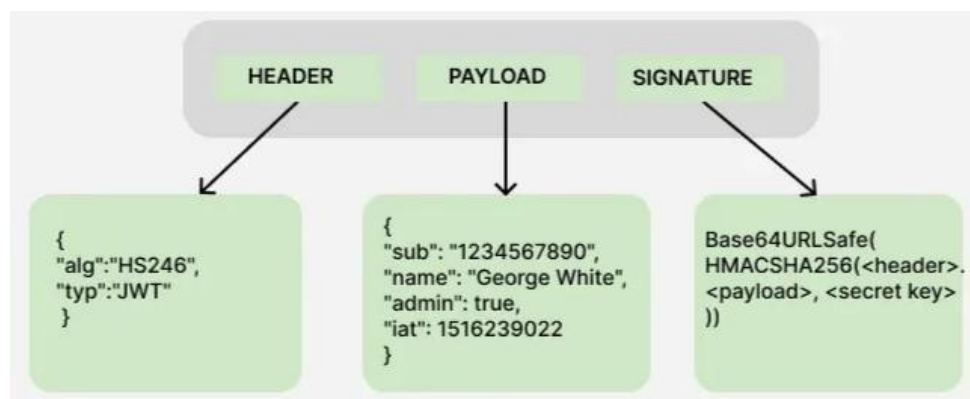


Figura 1: Estructura del token JWT

La firma solo previene de alteración de datos, ya que a los *JWT* se les conoce como tokens al portador (*Bearer Tokens*): quien tenga el *token* en su poder, tiene el acceso, sin importar quién lo envíe.

2. Implementación y flujo del uso en el *Login*

Como se muestra en la figura *Figura 2*, para usar *JWT* en el *Login*, se necesita que en el *backend* se implemente la creación del *token*, mientras que en el *frontend* se requiere definir el método de guardado del *token*.

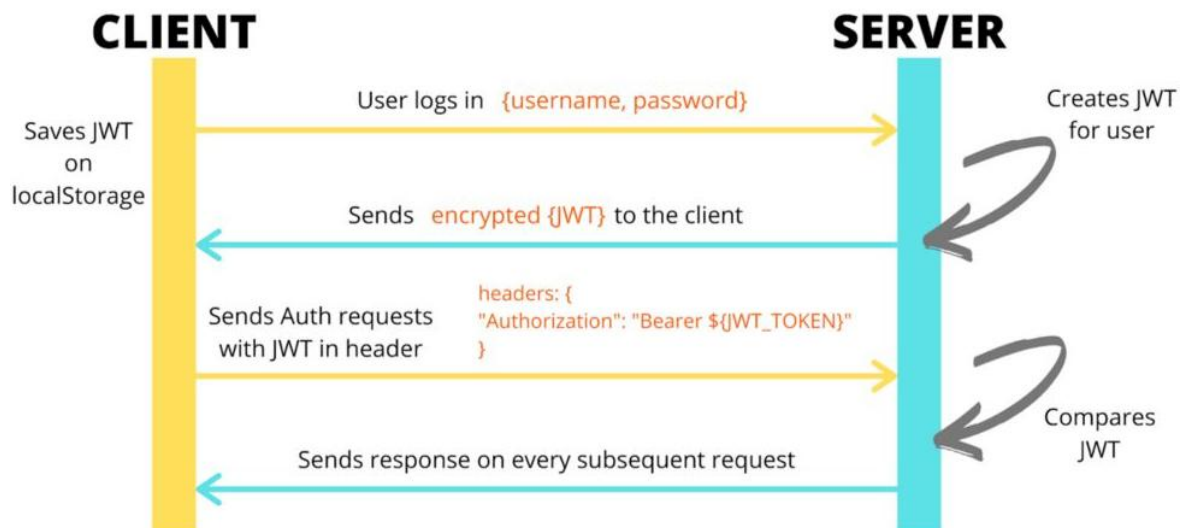


Figura 2: Flujo del Login y uso del JWT

La implementación del *Login* en el proyecto de clase *courseswap*, fue de la siguiente manera en el *backend*:

Como modelo (entidad) que representaba el usuario, solo se tuvo la tabla *estudiante*, ya que, no se tuvo manejo de roles, solo un único tipo de usuario :

```
@Table(name = "estudiante")
public class Estudiante {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "nombre", length = 100, nullable = false)
    private String nombre;

    @Column(name = "apellido", length = 100, nullable = false)
    private String apellido;

    @Column(name = "codigo", length = 7, nullable = false, unique = true)
    private String codigo;

    @Column(name = "email", length = 100, nullable = false, unique = true)
    private String email;

    @Column(name = "password", length = 100, nullable = false)
    private String password;

}
```

Luego se procedió con la creación del paquete *security* dentro del proyecto, donde se alojaron los *beans* o componentes principales :

- Configuración Global de Seguridad: En la clase *SecurityConfig* se definió la configuración principal anotada con *@EnableWebSecurity* para establecer las reglas de acceso a la API:
 - Deshabilitación de CSRF: Dado que la API es RESTful y Stateless, se desactivó la protección CSRF.
 - Gestión de Sesiones Sin Estado: Se configuró *SessionCreationPolicy.STATELESS* para garantizar que el servidor no guarde cookies ni sesiones HTTP.
 - Definición de Endpoints Públicos: Se liberaron las rutas del módulo de autenticación (*/api/auth/*) y la documentación de la API con *Swagger*. Todas las demás rutas de la aplicación requieren autenticación obligatoria.
 - Encriptación de Contraseñas: Se registró un bean *PasswordEncoder* utilizando *BCryptPasswordEncoder* para almacenar las claves de los estudiantes de forma segura.
 - Configuración de CORS: Se habilitó una política global de *CORS* para permitir solicitudes entre el cliente (Frontend) y el backend, configurando métodos HTTP permitidos (GET, POST, PUT, DELETE, etc.) y exponiendo el encabezado "Authorization"
 - Proveedor de Autenticación (*AuthenticationProvider*): Se configuró un *DaoAuthenticationProvider* al cual se le inyectó el servicio *CustomUserDetailsService* para la búsqueda del estudiante y el *BCryptPasswordEncoder* para verificar la contraseña ingresada en el login contra el hash de la base de datos.
 - Gestor de Autenticación (*AuthenticationManager*): Se registró el bean *AuthenticationManager* mediante la configuración de Spring Security, el cual sirve como punto de entrada principal para procesar y validar el inicio de sesión del estudiante en el controlador/servicio de autenticación.
 - Integración de Filtros Custom: Se registró el filtro de interceptación de JWT antes del filtro estándar de autenticación de usuario (*UsernamePasswordAuthenticationFilter*).

```

23
24 @Configuration @Jonkkeyler333 *
25 @EnableWebSecurity
26 public class SecurityConfig {
27
28     private final JwtAuthFilter jwtAuthFilter; 2 usages
29     private final CustomUserDetailsService userDetailsService; 2 usages
30
31     public SecurityConfig(JwtAuthFilter jwtAuthFilter, CustomUserDetailsService userDetailsService) { @Jonkkeyler333
32         this.jwtAuthFilter = jwtAuthFilter;
33         this.userDetailsService = userDetailsService;
34     }
35
36     @Bean @Jonkkeyler333
37     public PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }
40
41     @Bean @Jonkkeyler333
42     public AuthenticationProvider authenticationProvider() {
43         DaoAuthenticationProvider authProvider = new DaoAuthenticationProvider(userDetailsService);
44         authProvider.setPasswordEncoder(passwordEncoder());
45         return authProvider;
46     }
47
48     @Bean @Jonkkeyler333
49     public AuthenticationManager authenticationManager(AuthenticationConfiguration config) throws Exception {
50         return config.getAuthenticationManager();
51     }
52
53     @Bean @Jonkkeyler333
54     public SecurityFilterChain securityFilterChain(HttpSecurity http, CustomAuthenticationEntryPoint authEntryPoint) throws Exception {
55         http
56             .csrf(CsrfConfigurer<HttpSecurity> csrf -> csrf.disable())
57             .cors(CorsConfigurer<HttpSecurity> cors -> cors.configurationSource(corsConfigurationSource()))
58             .exceptionHandling(ExceptionHandlingConfigurer<HttpSecurity> exception -> exception.authenticationEntryPoint(authEntryPoint))
59             .authorizeHttpRequests(AuthorizationManagerRequestMat... auth -> auth
60                 .requestMatchers(@"/api/auth/**").permitAll()
61                 .requestMatchers(@"/api/web/**").permitAll()
62                 .requestMatchers(@"/v3/api-docs/**").permitAll()
63                 .requestMatchers(@"/swagger-ui/**").permitAll()
64                 .requestMatchers(@"/swagger-ui.html").permitAll()
65                 .anyRequest().authenticated()
66             )
67             .sessionManagement(SessionManagementConfigurer<HttpSecurity> session -> session
68                 .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
69             .authenticationProvider(authenticationProvider())
70             .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);
71
72         return http.build();
73     }
74
75     @Bean @Jonkkeyler333
76     public CorsConfigurationSource corsConfigurationSource() {
77         CorsConfiguration configuration = new CorsConfiguration();
78         configuration.setAllowedOriginPatterns(List.of("*"));
79
80         configuration.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS", "HEAD", "PATCH"));
81         configuration.setAllowedHeaders(List.of("Authorization", "Content-Type", "X-Requested-With", "Accept", "Origin"));
82         configuration.setExposedHeaders(List.of("Authorization")); // Para poder leer el JWT si viene en respuesta
83         configuration.setAllowCredentials(true);
84
85         UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
86         source.registerCorsConfiguration(pattern: "**", configuration);
87         return source;
88     }
89 }

```

- **Interceptación y Validación de Tokens:** Se realizó mediante la creación de la clase *JwtAuthFilter*, la cual tenía como objetivo ser un filtro personalizado que extiende de *OncePerRequestFilter* para procesar las peticiones HTTP entrantes mediante los siguientes métodos:
 - **Exclusión de Rutas Públicas:** En el método *shouldNotFilter* se especificó la omisión del filtro en los *endpoints* de Login que inician con */Auth* y documentación para optimizar el rendimiento al no hacer comprobaciones en *endpoints* que no requieren autenticación
 - **Filtrado:** Para los *endpoints* que si requieren la autenticación, en el método *doFilterInternal* se realiza la interceptación del encabezado *Authorization*, el filtro extrae el token JWT buscando el prefijo "Bearer ", en el *payload* del token se extrae el correo del estudiante (*username*) mediante el servicio *JwtService*. Si existe un usuario en el token y aún no hay una autenticación guardada en el contexto de seguridad actual (*SecurityContextHolder*), se valida la firma y expiración del token contra los detalles del usuario obtenidos por *CustomUserDetailsService*. Al verificar que el token es válido, se crea un objeto *UsernamePasswordAuthenticationToken* y se inyecta en el *SecurityContextHolder* de Spring Security, permitiendo que la solicitud continúe autorizada hacia los controladores

```

22  @Component  Jonkkeyler333
23  public class JwtAuthFilter extends OncePerRequestFilter {
24
25      private final JwtService jwtService; 3 usages
26      private final CustomUserDetailsService userDetailsService; 2 usages
27
28      public JwtAuthFilter(JwtService jwtService, CustomUserDetailsService userDetailsService) { Jonkkeyler333
29          this.jwtService = jwtService;
30          this.userDetailsService = userDetailsService;
31      }
32
33      @Override 2 usages Jonkkeyler333
34      protected boolean shouldNotFilter(HttpServletRequest request) throws ServletException {
35          // Omite la ejecución del filtro en los endpoints públicos de auth
36          return request.getServletPath().startsWith("/api/auth") || request.getServletPath().startsWith("/api/web") || request.getServletPat
37      }
38
39      @Override Jonkkeyler333
40      protected void doFilterInternal(
41          @NonNull HttpServletRequest request,
42          @NonNull HttpServletResponse response,
43          @NonNull FilterChain filterChain
44      ) throws ServletException, IOException {
45          final String authHeader = request.getHeader( name: "Authorization");
46
47          if (authHeader == null || !authHeader.startsWith("Bearer ")) {
48              filterChain.doFilter(request, response);
49              return;
50          }
51
52          final String token = authHeader.substring( beginIndex: 7);
53          final String email = jwtService.extractUsername(token);
54
55          if (email != null && SecurityContextHolder.getContext().getAuthentication() == null) {
56              UserDetails userDetailsService = userDetailsService.loadUserByUsername(email);
57              if (jwtService.isTokenValid(token, userDetailsService) ){
58                  UsernamePasswordAuthenticationToken authToken = new UsernamePasswordAuthenticationToken(userDetailsService, credentials: null, userDet
59                  authToken.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
60                  SecurityContextHolder.getContext().setAuthentication(authToken);
61              }
62          }
63          filterChain.doFilter(request, response);
64      }

```

En el paquete de servicios, *service*, se creó la clase *JwtService* donde se gestionan los tokens. Aquí se encuentra la lógica para la generación, firma, lectura y validación de los JSON Web Tokens (JWT) utilizados para autenticar las peticiones del estudiante. Fue anotada con *@Service*, esta clase encapsula el uso de la librería *JWT* (Java JWT). La clave secreta y el tiempo de expiración del token se obtienen mediante la anotación *@Value*

```
@Service 5 usages Jonkkeyler333
public class JwtService {

    @Value("NG96NXg4YXRkNG03Y3B3ZTFo0HowbTlsMjA5M3I4NzE=")
    private String secretKey;

    @Value("3600000")
    private long expirationTime;
```

La clase contiene los siguientes métodos:

- *getSignInKey()* Convierte la clave secreta configurada en formato *String* (Base64) a un arreglo de *bytes* utilizando el decodificador *Decoders.BASE64*. Posterior a esto, hace uso de *Keys.hmacShaKeyFor()* para generar una clave criptográfica simétrica segura (*SecretKey*) apta para el algoritmo de firma *HMAC-SHA*.

```
private SecretKey getSignInKey() { 2 usages Jonkkeyler333
    byte[] keyBytes = Decoders.BASE64.decode(secretKey);
    return Keys.hmacShaKeyFor(keyBytes);
}
```

- *generateToken(UserDetails userDetails)* Construye y firma un nuevo token JWT utilizando el patrón Builder de *JWT* (*Jwts.builder()*):
 - Define el '*subject*' usando el correo del estudiante (*userDetails.getUsername()*).
 - Registra la fecha y hora exacta de emisión mediante '*issuedAt*'.
 - Establece la fecha de expiración sumando el tiempo configurado (*expirationTime*) al tiempo actual mediante '*expiration*'.
 - Firma el token criptográficamente con la clave obtenida en *getSignInKey()* usando '*signWith*'.
 - Empaqueta y serializa el JWT en una cadena codificada lista para ser enviada al cliente con '*compact()*'.

```
24 @ public String generateToken(UserDetails userDetails) { 1 usage Jonkkeyler333
25     return Jwts.builder()
26         .subject(userDetails.getUsername())
27         .issuedAt(new Date(System.currentTimeMillis()))
28         .expiration(new Date(System.currentTimeMillis() + expirationTime))
29         .signWith(getSignInKey())
30         .compact();
31 }
```

- *extractAllClaims(String token)* Se encarga del desempaqueado y la validación de la firma del token

- *extractClaim(String token, Function<Claims, T> claimsResolver)*: Método genérico auxiliar que permite extraer cualquier información específica del Payload del token sin duplicar código. Toma el token y una función de resolución (Function<Claims, T>), obtiene todos los claims mediante *extractAllClaims()* y aplica la función recibida para retornar únicamente el dato solicitado.
- *extractUsername(String token)* Utiliza el método genérico *extractClaim* pasándole la referencia a método 'Claims::getSubject'. Retorna la identidad (correo del estudiante) que viene incrustada en el token.
- *extractExpiration(String token)* Hace uso de *extractClaim* junto con 'Claims::getExpiration' para obtener el timestamp de expiración registrado al momento de la emisión del token.
- *isTokenExpired(String token)* Compara la fecha de expiración obtenida del token con la fecha y hora actual del servidor (*new Date()*). Devuelve 'true' si el token ya venció o 'false' si aún se encuentra vigente.
- *isTokenValid(String token, UserDetails userDetails)* Realiza la comprobación final de seguridad para autorizar una petición HTTP. Compara que el usuario extraído del token coincida exactamente con el usuario cargado desde la base de datos (*userDetails.getUsername()*) y confirma que el token no haya expirado (*!isTokenExpired(token)*). Si ambas condiciones se cumplen, la autenticidad de la solicitud queda garantizada.

```

34      // Extrae el username (subject) del token
35      > public String extractUsername(String token) { return extractClaim(token, Claims::getSubject); }
36
37
38
39      // Verifica que el token pertenezca a este usuario y no haya expirado
40      @ public boolean isTokenValid(String token, UserDetails userDetails) { 1 usage  @ Jonkkeyler333
41          final String username = extractUsername(token);
42          return username.equals(userDetails.getUsername()) && !isTokenExpired(token);
43      }
44
45      > private boolean isTokenExpired(String token) { return extractExpiration(token).before(new Date()); }
46
47
48      > private Date extractExpiration(String token) { return extractClaim(token, Claims::getExpiration); }
49
50
51
52
53      @ private <T> T extractClaim(String token, Function<Claims, T> claimsResolver) { 2 usages  @ Jonkkeyler333
54          final Claims claims = extractAllClaims(token);
55          return claimsResolver.apply(claims);
56      }
57
58
59      private Claims extractAllClaims(String token) { 1 usage  @ Jonkkeyler333
60          return Jwts.parser().JwtParserBuilder
61              .verifyWith(getSignInKey())
62              .build().JwtParser
63              .parseSignedClaims(token).getPayload();
64      }

```

Finalmente con todas estas piezas, se contruyó el servicio que fue usado por el controlador encargado de la autorización. Este servicio fue *AuthService*, este servicio se le inyectaron las siguientes dependencias *EstudianteRepository*, *PasswordEncoder*, *JwtService* y *AuthenticationManager*

```
@Service 9 usages Jonkkeyler333
public class AuthService {
    private final EstudianteRepository estudianteRepository; 5 usages
    private final PasswordEncoder passwordEncoder; 2 usages
    private final JwtService jwtService; 2 usages
    private final AuthenticationManager authenticationManager; 2 usages

    public AuthService(EstudianteRepository estudianteRepository, PasswordEncoder passwordEncoder, JwtService jwtService, AuthenticationManager authenticationManager) {
        this.estudianteRepository = estudianteRepository;
        this.passwordEncoder = passwordEncoder;
        this.jwtService = jwtService;
        this.authenticationManager = authenticationManager;
    }
}
```

Este servicio, tenía los siguientes métodos:

- *registerUser(EstudianteRegisterDTO newEstudiante)*: Recibe un *DTO* validado con los campos necesarios para registrar un nuevo estudiante en la base de datos, validando que no se quisiera registrar un estudiante antiguo. Se crea un nuevo objeto *Estudiante*, que con ayuda del *passwordEncoder* se evita guardar la contraseña plana. Finalmente el repositorio guarda el nuevo objeto en la base de datos.

```
32 @ ~ public EstudianteResponseDTO registerUser(EstudianteRegisterDTO newEstudiante) { 1 usage Jonkkeyler333
33     ~ if (estudianteRepository.existsByEmail(newEstudiante.getEmail())) {
34         throw new IllegalArgumentException("El email ya está registrado");
35     }
36     var estudiante = new Estudiante();
37     estudiante.setEmail(newEstudiante.getEmail());
38     estudiante.setPassword(passwordEncoder.encode(newEstudiante.getPassword()));
39     estudiante.setNombre(newEstudiante.getNombre());
40     estudiante.setApellido(newEstudiante.getApellido());
41     estudiante.setCodigo(newEstudiante.getCodigo());
42     estudianteRepository.save(estudiante);
43     return EstudianteResponseDTO.builder()
44         .id(estudiante.getId())
45         .email(estudiante.getEmail())
46         .nombre(estudiante.getNombre())
47         .apellido(estudiante.getApellido())
48         .codigo(estudiante.getCodigo())
49         .build();
50 }
51
```

- *LoginUser(LoginRequestDTO loginRequest)* recibe el *DTO* validado de contraseña plano y usuario (correo), luego se le delega a la dependencia *AuthenticationManager*, que busque si existe un usuario con dicho correo y verifique la contraseña desenscriptando la contraseña que esta alojada en la base de datos, si todo sale bien, se busca el usuario mediante el repositorio y se crea el objeto con los detalles que necesita el servicio de los *tokens* para generarlo

```
52 @ public AuthResponseDTO loginUser(LoginRequestDTO loginRequest) { 1 usage Jonkkeyler333
53     authenticationManager.authenticate(new UsernamePasswordAuthenticationToken(loginRequest.getEmail(), loginRequest.getPassword()));
54     var estudiante = estudianteRepository.findByEmail(loginRequest.getEmail())
55         .orElseThrow(() -> new ResourceNotFoundException("Estudiante no encontrado"));
56     UserDetails userDetails = new User(
57         estudiante.getEmail(),
58         estudiante.getPassword(),
59         AuthorityUtils.NO_AUTHORITIES
60     );
61     String token = jwtService.generateToken(userDetails);
62     return new AuthResponseDTO(token, estudiante.getEmail());
63 }
```

Finalmente, en el paquete *controller*, se crea la clase *AuthController* el cual define las rutas REST en las cuales el usuario puede realizar el proceso de *Login* y el registro :

```

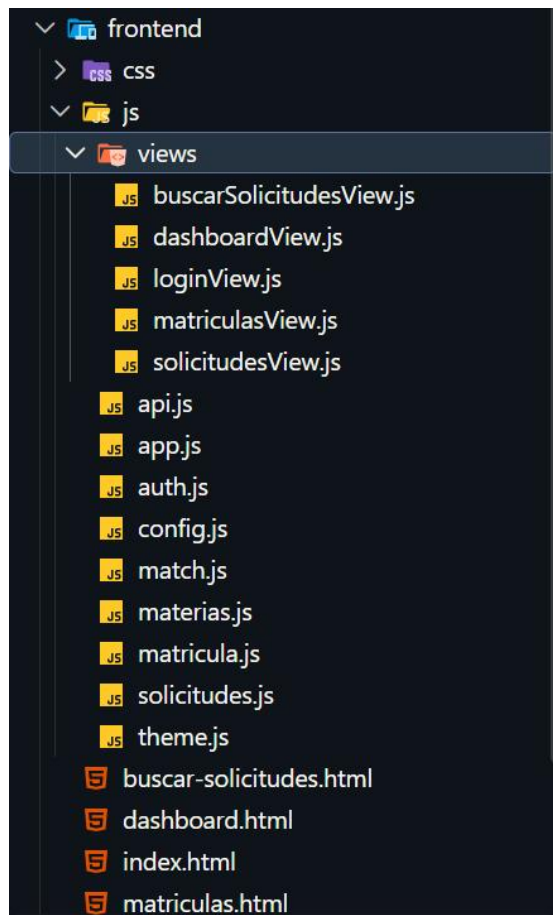
@RestController @ Jonkkeyler333
@RequestMapping("/api/auth")
@Tag(name = "Autenticación", description = "Endpoints de autenticación y registro de estudiantes")
public class AuthController {
    private final AuthService authService; 3 usages

    public AuthController(AuthService authService) { this.authService = authService; }

45     @PostMapping("/register") @ Jonkkeyler333
46     @Operation(summary = "Registrar un nuevo estudiante")
47     @ApiResponses(value = {
48         @ApiResponse(responseCode = "201", description = "Estudiante registrado correctamente",
49             content = @Content(schema = @Schema(implementation = EstudianteResponseDTO.class))),
50         @ApiResponse(responseCode = "400", description = "Datos inválidos o email ya registrado")
51     })
52     public ResponseEntity<EstudianteResponseDTO> registerUser(@Valid @RequestBody EstudianteRegisterDTO estudiante){
53         EstudianteResponseDTO estudianteResponseDTO = authService.registerUser(estudiante);
54         return ResponseEntity.status(HttpStatus.CREATED).body(estudianteResponseDTO);
55     }
56
57     @PostMapping("/login") @ Jonkkeyler333
58     @Operation(summary = "Iniciar sesión y obtener JWT")
59     @ApiResponses(value = {
60         @ApiResponse(responseCode = "200", description = "Autenticación exitosa",
61             content = @Content(schema = @Schema(implementation = AuthResponseDTO.class))),
62         @ApiResponse(responseCode = "400", description = "Credenciales inválidas")
63     })
64     public ResponseEntity<AuthResponseDTO> loginUser(@Valid @RequestBody LoginRequestDTO loginRequest) {
65         AuthResponseDTO authResponseDTO = authService.loginUser(loginRequest);
66         return ResponseEntity.ok(authResponseDTO);
67     }
68 }

```

En el *frontend* se realizó una separación de responsabilidades de los módulos JS, uno llamado *loginView.js* se encargaba de añadir los escuchadores de eventos a los botones de los formularios de inicio de sesión y de registro, mientras que el otro se encargaba de realizar el proceso de solicitud a los *endpoints* de autorización del *backend*, este fue llamado *auth.js*



El modulo *auth.js* , contiene el objeto *AuthService*, el cual, tiene los métodos asíncronos de *login*, *register* y *me*. En el proyecto se uso la API moderna de *Fetch* por lo cual se hizo manejo de las promesas mediante *await* y bloques *try-catch*. Además, al momento de realizar el *login* exitoso, se decidió guardar el token y la información del usuario en el *localStorage* de la sesión del navegador.

```
1 import { CONFIG } from "../config.js";
2 import { fetchAPI } from "../api.js";
3
4 export const AuthService = {
5   async login(email, password) {
6     const data = await fetchAPI("/auth/login", {
7       method: "POST",
8       body: JSON.stringify({ email, password }),
9     });
10    if (data.token) {
11      localStorage.setItem(CONFIG.TOKEN_KEY, data.token);
12    }
13    return data;
14  },
15
16  async register(userData) {
17    const data = await fetchAPI("/auth/register", {
18      method: "POST",
19      body: JSON.stringify(userData),
20    });
21    return data;
22  },
23
24  async me() {
25    const data = await fetchAPI("/estudiantes/me", {
26      method: "GET",
27    });
28    localStorage.setItem(CONFIG.USER_KEY, JSON.stringify(data));
29    return data;
30  },
31
32  logout() {
33    localStorage.removeItem(CONFIG.TOKEN_KEY);
34    localStorage.removeItem(CONFIG.USER_KEY);
35    window.location.href = "../index.html";
36  },
37
38  isAuthenticated() {
39    return !!localStorage.getItem(CONFIG.TOKEN_KEY);
40  },
41
42  checkAuthGuard(isProtectedRoute = true) {
43    const authenticated = this.isAuthenticated();
44    if (isProtectedRoute && !authenticated) {
45      window.location.href = "../index.html";
46    } else if (!isProtectedRoute && authenticated) {
47      window.location.href = "../dashboard.html";
48    }
49  },
50 };
51
```

Este objeto, luego en *loginView.js* para realizar las peticiones a los endpoints del *backend*, según corresponda si el usuario quiere realizar el login o desea registrarse:

```
1 import { AuthService } from "../auth.js";
2
3 export const initLoginView = () => {
4   AuthService.checkAuthGuard(false);
5
6   const loginForm = document.getElementById("login-form");
7   const alertMessage = document.getElementById("alert-message");
8   const registerForm = document.getElementById("register-form");
9
10  loginForm.addEventListener("submit", async (event) => {
11    event.preventDefault();
12    hideAlert();
13
14    const email = document.getElementById("login-email").value.trim();
15    const password = document.getElementById("login-password").value.trim();
16
17    try {
18      await AuthService.login(email, password)
19      window.location.href = './dashboard.html'
20    } catch (error) {
21      showAlert(error.message || 'Error al iniciar sesion', 'danger')
22      setTimeout(() => {
23        showAlert(null, null);
24        hideAlert();
25      }, 3500);
26    }
27  })
28
29  registerForm.addEventListener("submit", async (event) => {
30    event.preventDefault();
31    hideAlert();
32
33    const newUserData = {
34      nombre: document.getElementById("reg-nombre").value.trim(),
35      apellido: document.getElementById("reg-apellido").value.trim(),
36      codigo: document.getElementById("reg-codigo").value.trim(),
37      email: document.getElementById("reg-email").value.trim(),
38      password: document.getElementById("reg-password").value.trim()
39    }
40
41    try {
42      await AuthService.register(newUserData);
43      showAlert('Cuenta creada exitosamente. Inicia sesion para continuar.', 'success');
44      registerForm.reset();
45      const loginTab = new bootstrap.Tab(document.getElementById('tab-login'));
46      loginTab.show();
47    } catch (error) {
48      showAlert(error.message || 'Error al crear cuenta', 'danger');
49    }
50  })
51
52  function showAlert(msg, type = 'danger') {
53    alertMessage.textContent = msg;
54    alertMessage.className = `alert alert-${type} mt-3`;
55  }
56
57  function hideAlert() {
58    alertMessage.textContent = '';
59    alertMessage.className = 'alert d-none';
60  }
61
62
63 }
```